

1.4. Effacer le mot rangé sous un nom

Le compilateur connaît la longueur de MOT : on l'efface par son nom.

1. Ce qu'il faut comprendre

`effacer(5, 6, MOT)` efface les sept cases de `MOT`. Si l'on change le mot, l'effacement suit tout seul.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <effacer>` → [0.16.3. La fonction `effacer\(\)` — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **345 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         26 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
EffacerCases   12 o ← effacer() – efface des cases
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         96 o ← CopierTuiles – 6 lettres sur 43
"BONJOUR"      7 o ← texte() – un texte gravé
```

Lettres gravées : BJNORU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
effacer() × 1   8 o sur place, dans main() + EffacerCases (12 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte>    // écrit un texte à l'écran
#include <effacer>   // efface des cases, ou tout le fond
```

2. UN TEXTE NOMMÉ — MOT

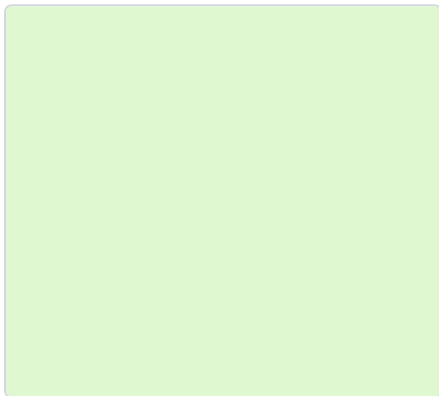
```
const char MOT[] = "BONJOUR";
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(5, 6, MOT);  
    effacer(5, 6, MOT);  
  
    while (true) image();  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

« BONJOUR » est écrit puis effacé : l'écran reste vide.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les sept cases de BONJOUR sont vides

✓ et l'écran est allumé

Ensuite : **1.5. La place du mot dans deux variables** — chapitre 1 : Les tout premiers pas. · gameboy3 — écrire une cartouche Game Boy en C++